Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 00 (2026) 000–000

Procedia Computer Science

www.elsevier.com/locate/procedia

30th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2026)

Closing the Triage Loop: MCP-mediated KLEE Validation of LLM-Reported C/C++ Vulnerabilities

Christopher Scherb[a,*], Alexander Trapp[a], Ruben Hutter[a], Nico Bachmann[a], Luc Bryan Heitz[a]

[a] *University of Applied Sciences and Arts Northwestern Switzerland, Bahnhofstrasse 6, 5210 Windisch, Switzerland*

Abstract

Large language models are proficient at flagging plausible vulnerability candidates in C/C++ code but cannot produce concrete proofs of exploitability. Symbolic execution engines like KLEE produce concrete crashing inputs but need a harness and the right preconditions. We close this loop with KLEE-MCP: a stateless Model Context Protocol (MCP) server that exposes KLEE as five tools (`auto_harness`, `verify_vulnerability`, `check_reachability`, `emit_reproducer`, `generate_harness`) to an LLM auditor, plus a Claude Code skill that drives the end-to-end workflow from a single natural-language instruction. The auditor emits structured candidates (CWE, tainted inputs, semantic bounds); the server auto-builds a KLEE harness, optionally links whole-library LLVM bitcode, either confirms the bug with a byte-accurate concrete input or proves it infeasible under the supplied bounds, and emits a disclosure-ready standalone reproducer (`reproduce.c` plus `poc.bin`) together with an exploitability classification and heuristic CVSS-3.1 vector. On 14 candidates spanning toy CWE programs and libpng 1.6.47 (full library linked as a 2.2 MB bitcode) the pipeline agrees with ground truth in 14/14 cases, and surfaces an unchecked precondition in `png_format_number` (`pngerror.c:140`) for which KLEE synthesises a concrete one-byte out-of-bounds write trigger in 4.2 s. All libpng-internal callers respect the function's contract, so this is a defense-in-depth finding rather than a remotely-reachable CVE. We additionally reproduce CVE-2015-8540 in `png_check_keyword`: KLEE synthesises a concrete crashing input in 1.26 s, the same out-of-bounds read is still reachable in libpng 1.6.47, and a counterfactual LLM bound that models the caller-side fix flips the verdict to infeasible. We argue that LLM-supplied semantic bounds encode the caller contract under which safety is being verified, not a performance knob, and demonstrate a self-correcting auto-relax retry when those bounds are wrong.

© 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the KES International.

Keywords: Vulnerability triage; symbolic execution; KLEE; large language models; Model Context Protocol; coordinated disclosure

1. Introduction

Triage is the bottleneck of C/C++ vulnerability work: static analysers emit thousands of candidates, most of which are false positives, and language models are good at proposing plausible candidates but cannot produce concrete crashing inputs. Symbolic execution — KLEE [1] being the standard reference — can produce such inputs if given

*Corresponding author. Email: christopher.scherb@fhnw.ch
1877-0509 © 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the KES International.

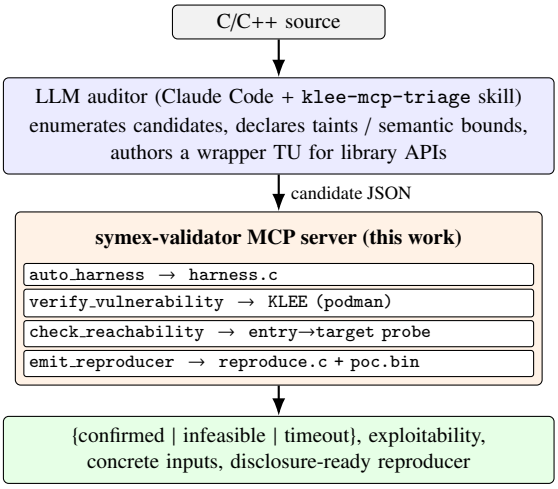

Fig. 1. KLEE-MCP pipeline. The LLM is the auditor; the MCP server is a stateless tool host around KLEE.

a harness with the right preconditions. We join these two halves through the Model Context Protocol (MCP) [13]: KLEE is exposed as an MCP server, and a tool-using LLM agent (Claude Code) commits to a structured candidate schema that the server turns into a harness and a verdict. The loop runs from a single natural-language instruction and closes with either a byte-accurate crashing input or a proof of infeasibility under the LLM's declared preconditions.

Contributions. **KLEE-MCP**[1] is a stateless MCP server exposing KLEE as five tools, plus a Claude Code skill that drives the full triage pipeline. Its *candidate schema* (CWE, tainted inputs, LLM-supplied semantic bounds, retry policy) turns LLM bounds into `klee_assume` predicates that the server can *automatically relax* on an infeasible verdict — surfacing, rather than hiding, cases where the LLM's precondition was wrong. For every confirmed bug the server emits a standalone `reproduce.c`, a `poc.bin`, and an exploitability classification (primitive, severity, region, CVSS-3.1 vector) aimed at a coordinated-disclosure workflow. As a proof-of-concept evaluation we run 14 candidates spanning toy CWE programs, bounded/unbounded sensitivity pairs, and libpng 1.6.47 as a whole-library 2.2 MB LLVM bitcode; 14/14 verdicts agree with ground truth. We further reproduce the CVE-2015-8540 pattern in `png_check_keyword`, confirming the same OOB in libpng 1.6.19 and 1.6.47 from the concrete byte-level trigger KLEE synthesises.

2. Design

Figure 1 shows the pipeline. The LLM auditor (Section 2.3) emits candidate JSON objects conforming to the candidate schema (Listing 1). Each candidate flows through the MCP server's tools (Section 2.2) and produces a structured verdict plus, for confirmed candidates, a reproducer and an exploitability classification.

2.1. Candidate schema

A candidate is a JSON object committing the LLM to a precise claim. Listing 1 shows an example. The mandatory fields are `source_file`, `function_name`, `cwe`. The remaining fields are optional refinements:

- `tainted_inputs` lists which parameters are attacker-controlled. If omitted, the server infers one symbolic taint per parameter from the function prototype.
- `assumptions` and `loop_bounds` carry LLM-supplied semantic bounds. They are delivered to KLEE as `klee_assume` calls and only applied when `use_bounds=true`. Example: the LLM knows that a size field in a network packet never exceeds 1500, so it emits `loop_bounds: {len: 1500}`.

[1] Source, skill, and reproducible benchmark: https://github.com/FHNW-Security-Lab/klee-mcp

- `auto_relax_on_infeasible`: if the bounded run returns `infeasible`, the server transparently retries without bounds and returns the relaxed verdict, annotated with `initial_verdict=infeasible` and a warning in the `notes`. This is what catches the case of *wrong* LLM bounds.
- `extra_bitcodes`: a list of pre-built LLVM `.bc` files that the runner `llvm-links` onto the harness before KLEE. This enables whole-library targets (Section 3).

Listing 1. Candidate JSON for the libpng `png_format_number` OOB write.

```
{"source_file": "examples/real/whole_lib_format_number.c",
 "function_name": "fuzz_format_number", "cwe": "CWE-787",
 "tainted_inputs": [
   {"name":"format", "c_type":"int", "size_bytes":4},
   {"name":"number", "c_type":"size_t", "size_bytes":8},
   {"name":"end_offset", "c_type":"unsigned int", "size_bytes":4}],
 "loop_bounds": {"end_offset":16, "number":256, "format":8},
 "use_bounds": true, "auto_relax_on_infeasible": true,
 "extra_bitcodes": ["realworld/bitcode/libpng.bc"],
 "timeout_s": 120, "max_memory_mb": 16000}
```

2.2. MCP tools

The server exposes five tools. `auto_harness` (*Phase 0*): given `source_file + function_name + cwe`, parse the function prototype, infer taints with sensible defaults (scalar / 256-byte buffer / NULL for `size_bytes=0`), and return the generated harness plus the inferred taint list. `verify_vulnerability` (*Phase 1*) is the main entry: it accepts the full candidate and runs KLEE inside `docker.io/klee/klee:3.1` under rootless podman, returning a `verdict` ∈ {confirmed, infeasible, timeout, build_failed, klee_error}, the concrete `.ktest` hex, per-symbolic-variable decoded values, and (when confirmed) an exploitability classification (Section 4). `check_reachability_tool` (*Phase 2*) takes an `entry_function` (an LLM-picked realistic API, not `main`) and a `target_function`, patches a scratch copy of the source to insert `klee_report_error` at the target, drives the entry function symbolically, and reports whether KLEE reaches the probe. `emit_reproducer` (*Phase 3*) writes a standalone `reproduce.c` inlining the KLEE-synthesised concrete values, a `poc.bin` of the raw bytes, and a gcc+ASan run recipe. `generate_harness_tool` is a manual escape hatch returning a harness from an explicit taint spec when auto-inference is inadequate.

2.3. Orchestrator skill

The Claude Code skill `klee-mcp-triage` is triggered by natural-language instructions such as "analyze `X` with klee-mcp". It enumerates source files, runs candidate hunting with a pattern-driven prompt, and then invokes the MCP tools per candidate in the order Phase 0 → Phase 1 → Phase 2 → Phase 3. For library APIs that require state setup (libpng's `png_create_read_struct`, libxml2's `xmlReadMemory`), the skill authors a small wrapper translation unit that exposes a simpler symbolic surface and forward-declares the library APIs it calls; the wrapper is linked against the library's whole-program bitcode at verify time via `extra_bitcodes`. A concrete example is the libpng push-mode harness used in Section 5: 50 lines of C that drives the full libpng push parser with 8 symbolic attacker bytes. The skill finally writes a single markdown report per run, with a summary table, per-confirmed-bug disclosure template (advisory header, root cause, reproduction, concrete inputs, suggested fix, coordinated-disclosure email boilerplate), rescued false positives, and unresolved candidates.

3. Implementation

KLEE [1] 3.1 and its runtimes (uClibc, POSIX) run inside a pinned container image with LLVM/clang 13, STP, and MiniSat; the server invokes it via rootless podman with `:Z` bind mounts, so no privileged operations are required on the host. Our function-level driving strategy is an instance of under-constrained symbolic execution [3].

For each tainted input the harness generator emits either `klee_make_symbolic` on a scalar or on a declared buffer (typed by the caller-supplied `c_type`, with a fallback for libpng-style typedef'd pointer types). LLM bounds are

Table 1. Exploitability primitives. The classifier combines KLEE's faulting-address expression (symbolic vs concrete) with the declared CWE and a source-line read/write heuristic to assign one of these labels to each confirmed verdict.

Primitive	Operation	Default severity
`arbitrary_write`	write, symbolic address	CRITICAL
`bounded_write`	write, concrete address	HIGH
`arbitrary_read`	read, symbolic address	HIGH
`bounded_read`	read, concrete address	MEDIUM
`uaf_read/write`	read/write on freed obj	HIGH
`null_deref`	read at 0	MEDIUM
`double_free`	free of freed ptr	HIGH
`integer_overflow`	arith, no direct mem eff	LOW
`div_by_zero`	arith	LOW
`crash`	abort/assert	LOW

emitted as `klee_assume` calls. A pointer taint with `size_bytes=0` means "pass NULL" — the idiom for exercising CWE-476 without symbolic pointer values.

For targets larger than a single translation unit, we ship `build_lib_bc.sh`: it compiles every `.c` in a source tree to `.bc` via clang in the KLEE container, then `llvm-links` them. On libpng 1.6.47 plus zlib 1.3.1 this produces a 2.2 MB combined bitcode in under one minute (30 TUs). The runner stages it alongside the harness before calling KLEE.

KLEE emits a `.err` file for *every* discovered error, including side effects that are not memory-safety bugs (libpng's own `abort()`, KLEE's internal `model/external` warnings). We therefore map each CWE to the KLEE error classes that actually confirm the bug and filter out the rest, logging the ignored errors for transparency. The reachability tool patches a copy of the source to insert `klee_report_error` at the target, then runs KLEE with `--exit-on-error-type=ReportError` so the first reaching path aborts cleanly and the entry-function input becomes the reachability witness.

4. Exploitability classification

A key deliverable for a coordinated-disclosure workflow is *severity context*: is this crash an arbitrary write an attacker can weaponise, or is it a bounded denial-of-service? We add a heuristic classifier (Table 1) that runs after every confirmed verdict. It consumes (i) the KLEE `.err` file's `Info:` section, (ii) the candidate's declared CWE, and (iii) the source line at the sink.

The *symbolic-address* flag is the key distinguishing signal. A KLEE `.err Info:` block contains an `address:` line; if the value is a K-Query expression (e.g., `(Add w64 X (ReadLSB w64 0 whereami))`), the attacker controls which address is dereferenced and the primitive is the *arbitrary* variant. A concrete integer means KLEE resolved the address to a specific location, so the primitive is *bounded*.

The classifier also extracts (i) the target memory region (`stack` / `heap` / `static`) from KLEE's allocation note (`MO... allocated at ...: %x = alloca i32` vs `malloc`), with a CWE-driven fallback, and (ii) a *program-counter-overwrite possibility* flag that is `true` for arbitrary writes and for stack-located bounded writes. A heuristic CVSS-3.1 base vector is attached for disclosure-template prefill (the paper uses these values verbatim in Section 5; they should be refined per engagement context).

5. Evaluation

5.1. Setup

All runs use the same environment: Arch Linux host, rootless podman 5.8.2, `docker.io/klee/klee:3.1` (KLEE 3.1, LLVM/clang 13, STP + MiniSat). Each candidate is capped at 16 GB RSS and a per-candidate wall-

Table 2. 14-candidate benchmark. B = `use_bounds=true`; R = server auto-relaxed after a bounded infeasible run. All verdicts match ground truth.

Candidate	CWE	Exp.	Verdict	B/R	Wall	Primitive / severity
Phase 1 — toy examples						
bof_01	121	c.	c.	–	1.0 s	bounded_write / HIGH
intoverflow_01	190	c.	c.	–	2.4 s	bounded_write / HIGH
null_01	476	c.	c.	–	0.9 s	null_deref / MED
uaf_01	416	c.	c.	–	0.9 s	uaf_read / HIGH
safe_01 (neg.)	121	i.	i.	–	1.1 s	—
Phase 2 — LLM bounds & auto-relax						
bof_01_bounded	121	c.	c.	B	0.9 s	bounded_write / HIGH
bof_01_tootight	121	c.	c.	B R	0.9 s	bounded_write / HIGH
Phase 3a — extracted real libpng						
libpng_fp_number	125	i.	i.	B	30.9 s	—
libpng_fp_number_bug	125	c.	c.	B	5.9 s	bounded_read / MED
libpng_fp_number_unb.	125	c.	c.	–	9.4 s	**arbitrary_read** / HIGH
Phase 3b — whole-library libpng+zlib bitcode						
libpng_sig_cmp	125	i.	i.	B	5.1 s	—
libpng_fp_whole	125	i.	i.	B	42.9 s	—
libpng_push_read	OTH	i.	i.	B	4.1 s	—
libpng_format_number	787	c.	c.	B	4.2 s	arbitrary_write[†]

[†] Contract-violation finding, not internally reachable in libpng 1.6.47; see §5.

time budget (30–180 s). A single command (`python benchmark/run_bench.py`) runs all 14 candidates and writes `benchmark/results.csv`.

5.2. Benchmark

Table 2 lists all 14 candidates. Ground-truth verdicts are encoded in each candidate JSON (`expected_verdict`). The benchmark covers five phases: (1) toy programs exercising each target CWE, (2) LLM bounds + auto-relax, (3) reachability, (4) extracted real libpng functions (self-contained TU), and (5) whole-library libpng+zlib bitcode.

Result: 14/14 verdicts match ground truth. Median wall time 2.4 s; worst case 42.9 s on the whole-library `png_check_fp_number` exhaustion. Memory usage stayed well below the 16 GB cap on every row. Phase-1 and -2 rows complete sub-second; extracted libpng targets (P3a) take 6–31 s; whole-library linked targets (P3b) 4–43 s.

Confirmed-vs-infeasible separation (C1). Seven rows return `confirmed` with a byte-accurate KLEE-synthesised `.ktest`; the remaining seven return `infeasible` with KLEE's `done: completed paths = N, partially completed = 0` — proofs under the declared bound. The sensitivity pair `libpng_fp_number` (bounded, infeasible in 30.9 s, 1438 completed paths) vs `libpng_fp_number_unb.` (unbounded, confirmed in 9.4 s) shows both verdicts are correct under their respective contracts: the bounded run proves the loop safe *given* a truthful `size`; the unbounded run shows the function trusts `size` blindly, so a caller over-reporting `size` causes an OOB read. Bounds therefore encode the *precondition* under which safety is being verified, not a performance knob.

LLM bounds as caller contracts, self-correcting (C2). Row `bof_01_tootight` injects a deliberate LLM error: `loop_bounds={len: 10}` for a bug that triggers at `len > 16`. The bounded run returns `infeasible`; the server auto-relaxes and returns `confirmed` in 0.9 s with `initial_verdict=infeasible`, `relaxed_retry_performed=true`, and a notes field "*LLM bounds ruled out the crashing input*" — the pipeline surfaces LLM mis-modelling rather than silently mis-verifying. On real-library targets the speedup is load-bearing: an unbounded run of the libpng FP state machine timed out at 60 s with 1700 partial paths; with `loop_bounds={size: 4}` the same run exhausts 1438 complete paths in 30.9 s.

Historical CVE reproduction: CVE-2015-8540 [16]. We extracted `png_check_keyword` from libpng 1.6.19 (last release before the 1.6.20 fix) into a byte-identical standalone TU, driven with a 4-byte symbolic non-null-terminated `key` and an 80-byte output buffer. KLEE **confirmed the OOB read at line 43 in 1.26 s**, synthesising `key = 0x01 0x01 0x01 0x01`: the `while (*key && key_len < 79)` loop reads `key[4]`, one byte past the allocation. Extracting the same function from libpng 1.6.47 (body preserved verbatim, moved to pngset.c) and running the identical

harness yields **the same OOB at line 34 in 1.26 s with the identical trigger**; the 1.6.20 fix lived in callers (null-terminating before invocation), not inside `png_check_keyword`. Adding the counterfactual LLM bound `key[3] == 0` (modelling the caller contract) flips the verdict to **infeasible in 1.17 s after exhausting 103 completed paths**, directly illustrating on a real CVE that LLM bounds encode caller contracts and that the same function is safe-under-contract and unsafe-off-contract.

Cross-library audit. A focused libpng tRNS chunk-handler harness (valid prefix, symbolic length and payload) timed out at 240 s — real handler exercised, no matching error. A libjpeg-turbo 3.0.4 harness (whole-library 3.5 MB bitcode) through `jpeg_read_header` returned infeasible in 5.4 s but only 1 completed path: KLEE 3.1 does not model the `longjmp`-based error paths, so the first malformed marker collapses exploration. A libxml2 2.12.6 harness (9.4 MB bitcode, 106 TUs) calling `xmlReadMemory` on 32 symbolic bytes returned infeasible in 23 s with **11 completed paths and 523 K LLVM instructions executed** — an end-to-end second-library result. Where the pipeline does not find a bug, the failure mode is an engine-side limit we can characterise.

Whole-library bitcode scales to real code (C3). Rows 11–14 link against a single 2.2 MB `libpng+zlib.bc` built once from 30 TUs. Row 13 (`libpng_push_read`) is the realistic fuzzer-style entry: a 50-line wrapper creates a `png_struct` via `png_create_read_struct`, registers callbacks, and feeds 8 symbolic bytes through `png_process_data`; KLEE returns `infeasible` in 4.1 s under libpng's `setjmp`/`longjmp` error handling (with the runner filtering libpng's own `abort()` side effects).

Row 14 (`libpng_format_number`) illustrates both the kind of finding the pipeline surfaces automatically and the care required in reporting it. `pngerror.c:140` decrements `end` and writes a null terminator unconditionally, before any check against `start`. KLEE synthesised the minimum contract-violating input (`end_offset=0`, i.e. `end == start`) in 4.2 s; the classifier labels the resulting one-byte OOB `arbitrary_write` because the address is symbolic, and the auto-generated `reproduce.c` reproduces the ASan report. *Scope, honestly:* the function's doc-comment requires `end > start` and it is marked `PNG_INTERNAL_FUNCTION`. We grepped every call site in libpng 1.6.47; all of them pass `buffer + sizeof(buffer)` as `end`, so the OOB is not reachable from any code path shipped with libpng. Row 14 is a *defense-in-depth* observation (an unchecked precondition a future refactor or a third-party consumer could violate), not a remotely-reachable CVE. A two-line defensive patch (`if (end <= start) return end;`) is drafted for coordinated disclosure; at submission time the libpng maintainers had not yet responded. The method claim here is about the pipeline's ability to surface such contract violations automatically, not about the severity of this specific bug.

6. Related work

Harness synthesis with an LLM. SymTEE [7] prompts an LLM to emit KLEE-compatible harnesses with mock environments for trusted-execution-environment applications; PromptFuzz [5] and OSS-Fuzz-Gen [6] synthesise drivers for coverage-guided fuzzers. Our setting is broader in scope (general C/C++ whole-library bitcode) and different in interface: KLEE is exposed as an MCP server, the LLM commits to a JSON candidate schema with semantic bounds that become retractable `klee_assume` predicates, and every confirmed bug carries a standalone reproducer plus an exploitability classification.

LLM embedded in a symbolic engine. ConcoLLMic [8] and COTTONTAIL [9] place the LLM inside a concolic engine as a constraint solver or symbolizer, reporting 115–233 % branch-coverage gains over KLEE and new CVEs; AutoBug [10] replaces the SMT solver entirely. Our design keeps KLEE as the authoritative oracle and places the LLM outside it, trading coverage gains for a mechanical verdict under explicit LLM-declared preconditions.

Automated PoC generation. PoCGen [11] targets npm CVEs with LLM + static taint + dynamic validation; DrillAgent [12] emits PoVs via sanitizer-guided agents. Both use concrete execution as the oracle. We target C/C++ and use the symbolic-execution witness to emit a standalone reproducer with an exploitability and CVSS-3.1 classification oriented at coordinated disclosure.

Foundations and MCP as substrate. KLEE [1], angr [2], under-constrained symex [3], and Fudge [4] establish the symbolic-execution and harness-synthesis stack this work sits on. Our prior work [14, 15] pushes symex on the engine side; KLEE-MCP addresses the complementary driver-side question of where candidates, harnesses, and preconditions come from, and treats the engine as an exchangeable oracle behind an MCP [13] interface — a protocol that the existing literature mostly studies as a threat surface, not an analysis substrate.

7. Limitations, future work, and conclusion

KLEE-MCP is fast when LLM bounds are reasonable and the target function has a small enough symbolic surface. Without bounds, real-library parsers time out; we consider this acceptable because well-chosen semantic bounds are exactly what an LLM is best at supplying. Whole-library bitcode handles libraries in the libpng / libxml2 class; scaling to codebases with dynamic linking, heavy C++ templates, or inline assembly (libjpeg-turbo's SIMD paths) requires engineering we do not yet report. The heuristic exploitability classifier does *not* prove a working exploit — it reports the primitive class the crash exposes. Our single non-synthetic finding (Row 14) is a defense-in-depth contract violation rather than a remotely-reachable CVE, and we were unable to obtain maintainer feedback before the submission deadline; the paper's claim is therefore about the pipeline's *method*, not about a body of externally-acknowledged vulnerabilities. Our cross-library audit (libpng-tRNS focused, libjpeg-turbo, libxml2) showed the pipeline scales end-to-end on a second whole-library target (libxml2, 11 exhaustively-completed parser paths in 23 s) but that stock KLEE's `setjmp`/`longjmp` handling and state compaction become the binding constraint on deeper image-format targets. Verifying program-counter overwrite, adding `setjmp`-aware symbolic execution, and running historical CVEs are the obvious follow-ons.

An MCP server is a practical interface for exposing symbolic execution to an LLM auditor, and the combination validates and reproduces real-library vulnerabilities and contract violations while cleanly separating confirmed bugs from rescued false positives. The methodological insight worth generalising beyond KLEE is that LLM-supplied bounds encode *caller contracts*, and a mechanical auto-relax retry catches when the contract is wrong. We release the server, skill, benchmark, and all 14 candidates at https://github.com/FHNW-Security-Lab/klee-mcp.

Acknowledgements

This work was carried out at the University of Applied Sciences and Arts Northwestern Switzerland.

References

[1] Cadar, C., Dunbar, D., Engler, D. (2008) "KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs." In *OSDI*.
[2] Shoshitaishvili, Y., et al. (2016) "SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis." In *IEEE Symposium on Security and Privacy*.
[3] Ramos, D. A., Engler, D. (2015) "Under-Constrained Symbolic Execution: Correctness Checking for Real Code." In *USENIX Security*.
[4] Babić, D., et al. (2019) "FUDGE: Fuzz Driver Generation at Scale." In *ESEC/FSE*.
[5] Lyu, Y., Xie, Y., Chen, P., Chen, H. (2024) "Prompt Fuzzing for Fuzz Driver Generation." In *ACM CCS*. arXiv:2312.17677. https://doi.org/10.1145/3658644.3670396.
[6] Google. "OSS-Fuzz-Gen — LLM-driven fuzz-driver generation." Software repository, https://github.com/google/oss-fuzz-gen (accessed 2026-04-19).
[7] Ma, C., Shi, J., Han, R., Liu, Y., Niu, Y., Lo, D. (2026) "Finding Missing Input Validation in TEEs via LLM-Assisted Symbolic Execution." In *FORGE 2026*.
[8] Luo, Z., Zhao, H., Wolff, D., Cadar, C., Roychoudhury, A. (2026) "Agentic Concolic Execution." In *IEEE Symposium on Security and Privacy*.
[9] Tu, H., Lee, S., Li, Y., Chen, P., Jiang, L., Böhme, M. (2026) "Cottontail: Large Language Model-Driven Concolic Execution for Highly Structured Test Input Generation." In *IEEE Symposium on Security and Privacy*. arXiv:2504.17542.
[10] Li, Y., Meng, R., Duck, G. J. (2025) "Large Language Model Powered Symbolic Execution." In *OOPSLA*. arXiv:2505.13452.
[11] Simsek, D., Eghbali, A., Pradel, M. (2025) "PoCGen: Generating Proof-of-Concept Exploits for Vulnerabilities in npm Packages." arXiv:2506.04962.
[12] Li, H., Che, X., Wang, Y., Liao, X., Xing, L. (2026) "Execution-State-Aware LLM Reasoning for Automated Proof-of-Vulnerability Generation." arXiv:2602.13574.

[13] "Model Context Protocol Specification, revision 2025-11-25." https://modelcontextprotocol.io/specification/2025-11-25 (accessed 2026-04-19).

[14] Scherb, C., Heitz, L. B., Grieder, H. (2024) "Divide and Conquer Based Symbolic Vulnerability Detection." arXiv:2409.13478.

[15] Scherb, C., Jahn, M., Trapp, A., Bachmann, N., Hutter, R., Heitz, L. B. (2026) "Identifying Critical Failure Chains for Resilience in Cyber-Physical Systems using Symbolic Execution." In *IEEE Consumer Communications & Networking Conference (CCNC)*. https://doi.org/10.1109/CCNC65079.2026.11366287.

[16] NIST NVD. "CVE-2015-8540 — libpng out-of-bounds read in `png_check_keyword`." https://nvd.nist.gov/vuln/detail/CVE-2015-8540 (accessed 2026-04-19).